

CPU调度

中国人民大学信息学院

卞昊穹

bianhq@ruc.edu.cn

内部资料，请勿外传

Outline

- CPU调度基本策略
- Multi-Level Feedback Queue
- Proportional Share
- 多核CPU调度

CPU调度

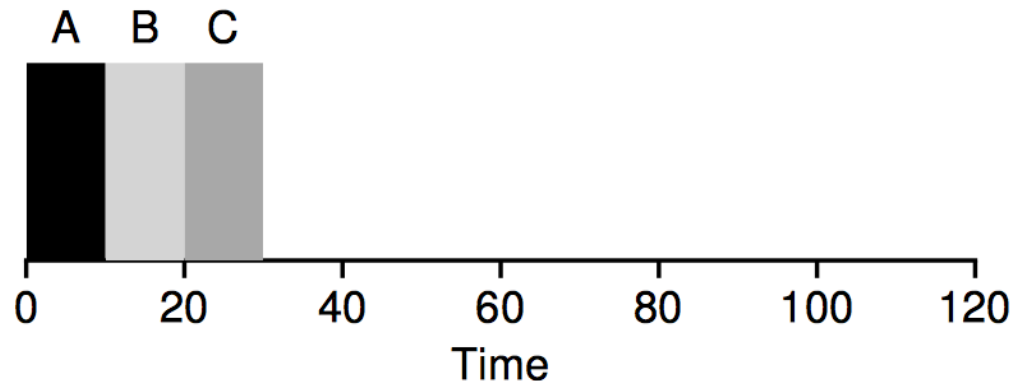
- 中断/异常：进程切换的方法
- 调度：进程切换的决策
 - 优化目标1：更小的**turnaround time**（周转时间），任务从到达整体完成的延迟

$$T_{turnaround} = T_{completion} - T_{arrival}$$

- 多任务：尽可能小的平均**turnaround time**

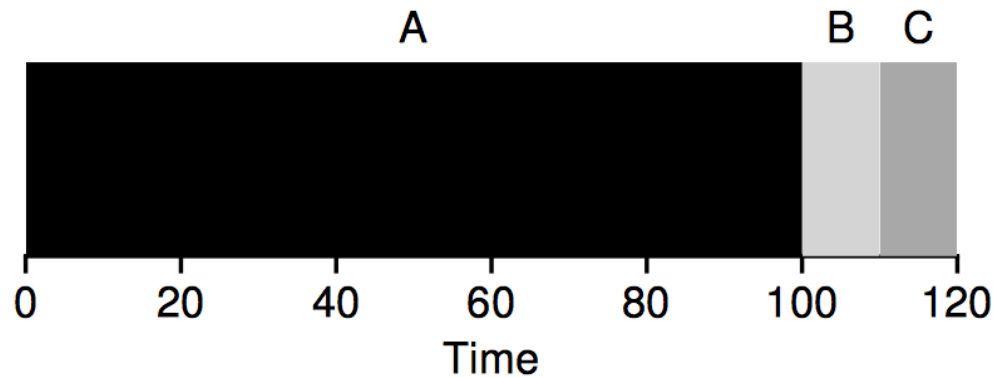
First In, First Out (FIFO)

- First Come, First Served (FCFS)
- 例如A, B, C几乎同时到达, $T_{\text{arrival}} = 0$
- 平均turnaround time = $(10+20+30)/3=20$
- 优点: 简单



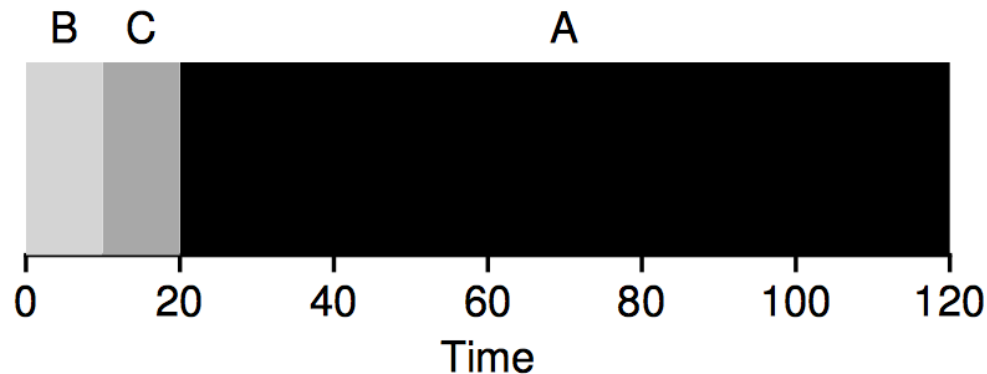
First In, First Out (FIFO)

- 缺点
 - 当任务长度差距较大时，如下图
 - 平均turnaround time = $(100+110+120)/3 = 110$



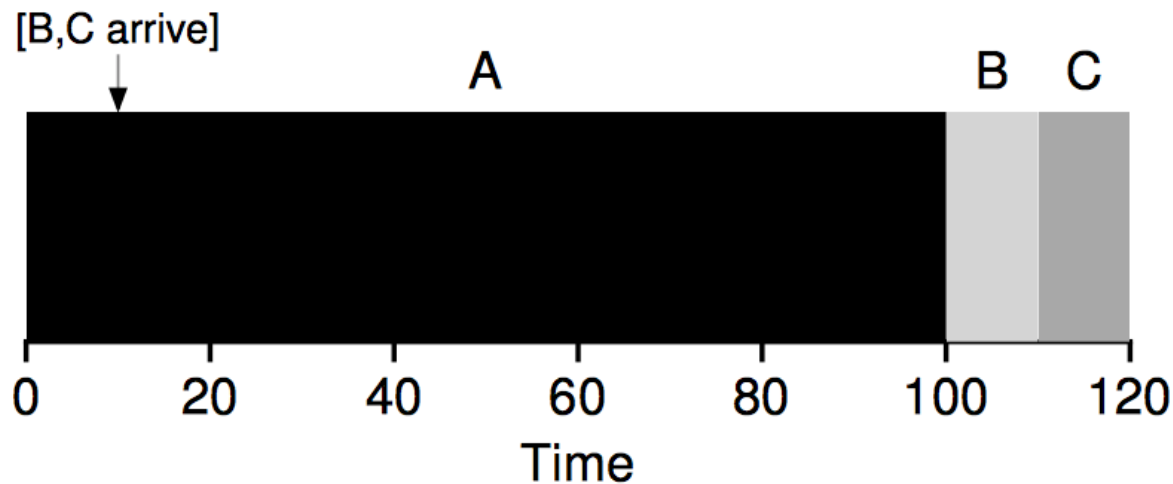
Shortest Job First (SJF)

- 有助于减小平均turnaround time, 例如上面的情况
- 平均turnaround time = $(10+20+120)/3 = 50$



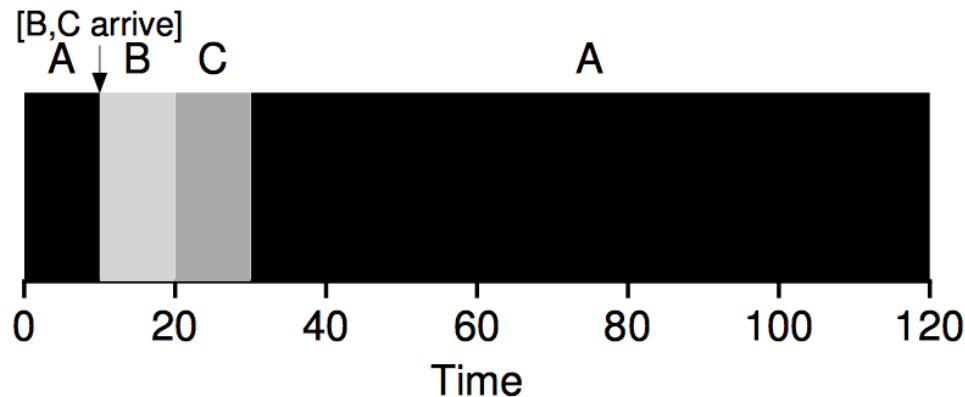
Shortest Job First (SJF)

- 对于任务同时到达， SJF是最优调度算法了
- 但如果长任务A先到达， 则SJF没有办法
 - 例如A在0时刻到达， B和C在10时刻到达
 - 平均Turnaround time = $(100+100+110)/3 = 103.3$



Shortest Time-to-Completion First (STCF)

- 上述问题改进方法：
 - 不要把一个任务执行到底才切换
 - 允许任务抢占（时间片/中断作为切换点）
- STCF
 - 在SJF上增加抢占功能，剩余时间最短的任务优先
 - 平均turnaround time = $(120+10+20)/3 = 50$



平均Turnaround Time的问题

- 平均Turnaround Time反映整体性能
 - 但有些Job被平均（自己牺牲，换来整体平均更好，例如长任务）
- 不能做到公平（Fairness）
 - 用户感受可能不好
 - 极端情况出现“饥饿问题”（job不断到达，某些Job一直不被调度）
- 如果所有job属于一个用户，还可以接受
 - 如果jobs属于不同用户（尤其是付费用户），则无法接受牺牲
 - 云计算下的公平性问题
 - 效率和公平往往不能兼得

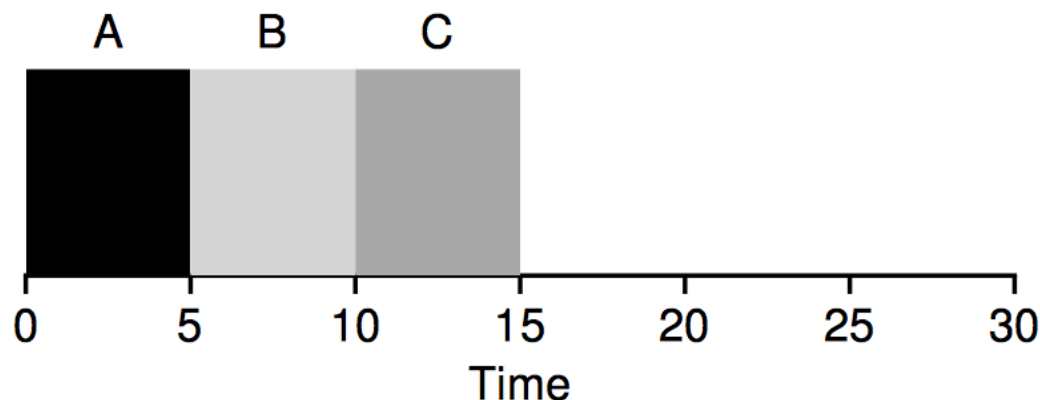
CPU调度

- 优化目标2: **Response time**

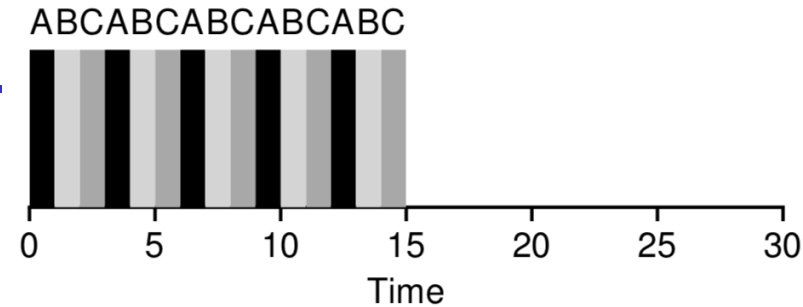
- 任务的**第一次被调度时间**与到达时间之差

$$T_{response} = T_{firstrun} - T_{arrival}$$

- 之前的调度算法（如**SJF**）会导致**response time**很差（**B**和**C**）



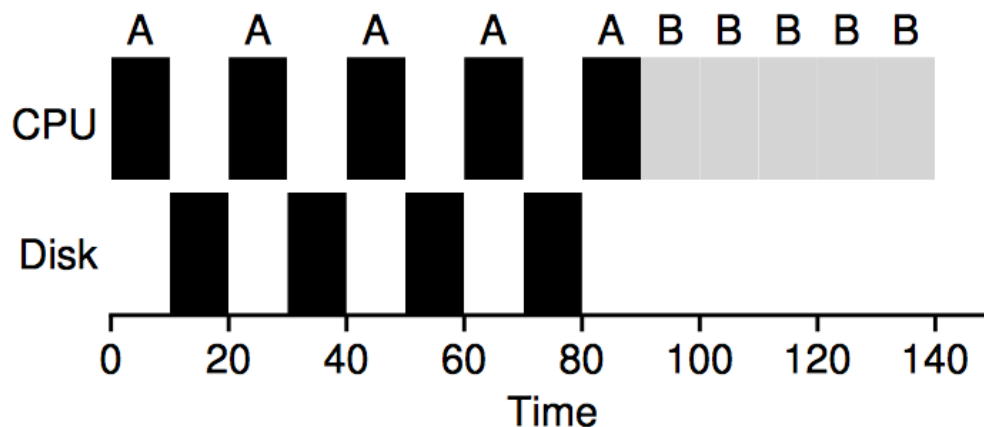
Round Robin (RR)



- 基于时间片，轮流执行任务
 - **response time**小
 - 增加**context switch**的开销
 - 增大时间片能减小开销的比例，但也会增大**response time**
 - 所以需要在二者之间**tradeoff**
 - 偏重公平(response time): RR
 - 不看重公平(turnaround time): SJF, STCF
- **Systems**领域多数情况都是在**tradeoff**，因为实际情况因素很多，某方面最优一般都伴随其他方面的副作用

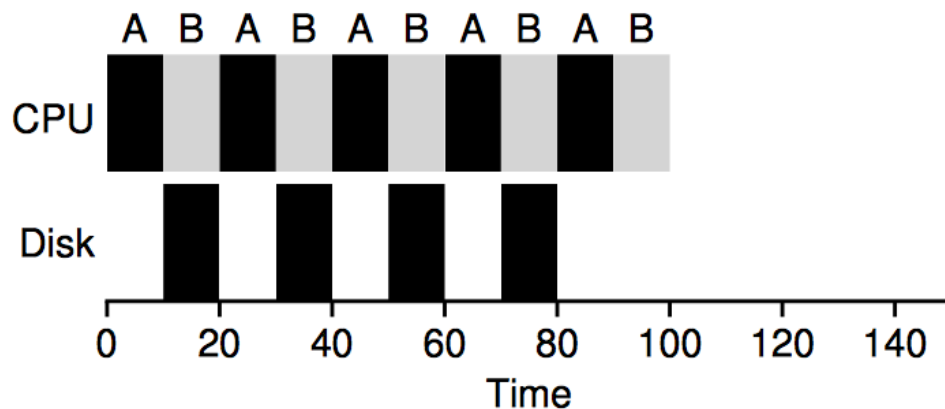
考虑I/O

- I/O操作远比CPU周期要慢
- 进程执行I/O操作时，要block住，直到I/O完成
- STCF的问题
 - A和B都是50ms的任务
 - 每次interrupt，A剩余的时间小于B，那么会优先调度A，浪费CPU时间



考虑I/O

- 解决方案
 - 把A看做5个10ms的子任务
 - 每次A开始I/O时，调度器只剩B一个选项，调度B
 - 当I/O完成时（**interrupt**），A的剩余时间短，调度A
 - （或者增加调度条件，有I/O阻塞不能调度）



课堂练习

例 1：假定要在一台处理器上执行如下图所示的作业，它们在 0 时刻以 1, 2, 3, 4, 5 的顺序到达。给出采用下列调度算法时的调度顺序、平均周转时间(turnaround time)和平均响应时间(response time)

(1) FCFS

(2) RR(时间片为 1, 不考虑优先级)

(3) 非抢占式 SJF(shortest job first)

作业	执行时间	优先级
1	10	3
2	1	1
3	2	2
4	3	4
5	5	2

练习答案

作业	执行时间	优先级
1	10	3
2	1	1
3	2	2
4	3	4
5	5	2

(1) FCFS: (2 分)

P1	P2	P3	P4	P5	
0	10	11	13	16	21

$$\text{平均响应时间} = (0 + 10 + 11 + 13 + 16) / 5 = 10$$

$$\text{平均周转时间} = (10 + 11 + 13 + 16 + 21) / 5 = 14.2$$

(2) RR(TQ=1)

P1	P2	P3	P4	P5	P1	P3	P4	P5	P1	P4	P5	P1	P5	P1	P5	P1	P1	P1	P1	P1	
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21

$$\text{平均响应时间} = (0 + 1 + 2 + 3 + 4) / 5 = 2$$

$$\text{平均周转时间} = (21 + 2 + 7 + 11 + 16) / 5 = 11.4$$

练习答案

作业	执行时间	优先级
1	10	3
2	1	1
3	2	2
4	3	4
5	5	2

(3) SJF

P2	P3	P4	P5	P1
0	1	3	6	11
				21

$$\text{平均响应时间} = (11 + 0 + 1 + 3 + 6) / 5 = 4$$

$$\text{平均周转时间} = (21 + 1 + 3 + 6 + 11) / 5 = 8.4$$

课堂练习

- 有5个待运行作业，估计它们的运行时间分别是9、6、3、5、X。以何种顺序运行这些作业能够得到最短的平均响应时间（答案将依赖于X）。

练习答案

- 其他四个作业的顺序应该从短到长：
- 3、5、6、9
- 那么X可能处于5种位置

Outline

- CPU调度基本策略
- **Multi-Level Feedback Queue**
- Proportional Share
- 多核CPU调度

以上调度算法的共同问题

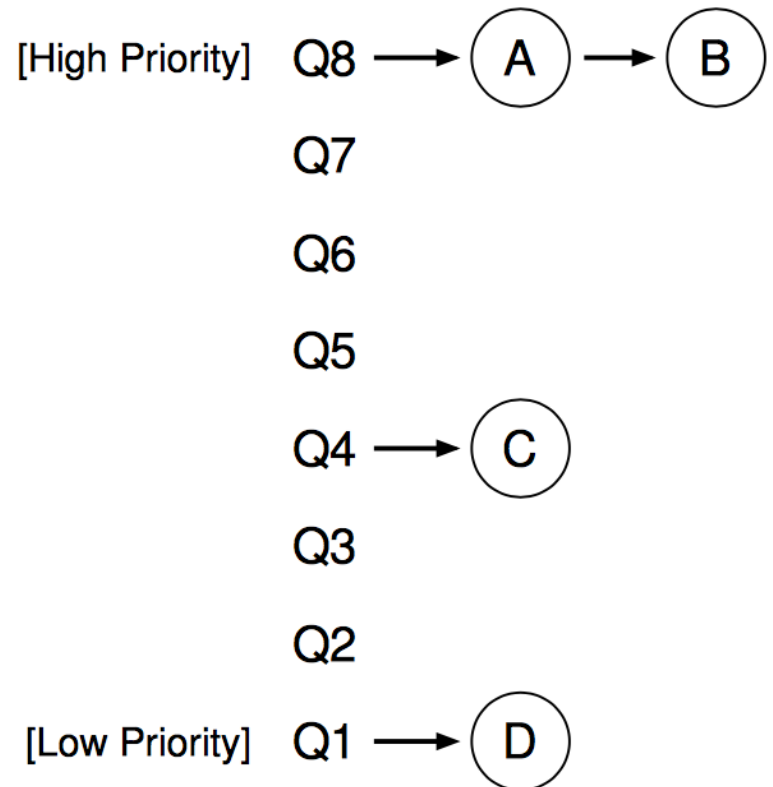
- 假设我们知道每个任务的时间，从而短任务优先
- 但实际上我们一般是没法预知的

Multi-Level Feedback Queue (MLFQ)

- 提出者
 - Corbato, 1962, Compatible Time-Sharing System (CTSS) 的一部分
 - Corbato后来参与Multics操作系统的工作
 - 后来获得ACM图灵奖
- MLFQ目标
 - 不知道任务长度的前提下，尽量短任务优先来减小turnaround time
 - 减小response time，但又不像RR那样伤害turnaround time

Multi-Level Feedback Queue (MLFQ)

- 分为多个优先级队列
- 规则
 - **Rule 1:** If $\text{Priority}(A) > \text{Priority}(B)$, A runs (B doesn't).
 - **Rule 2:** If $\text{Priority}(A) = \text{Priority}(B)$, A & B run in RR.



Multi-Level Feedback Queue (MLFQ)

- **Workload**分析

- 一种是interactive short-running任务，经常交换，让出CPU，对response time要求高
- 一种是long-running CPU-bound任务，对response time要求不高，但对turnaround time要求高

- **System**优秀论文经常这样，抓住主要矛盾（通过workload分析等方式），针对性设计（传统方法一般不这样）

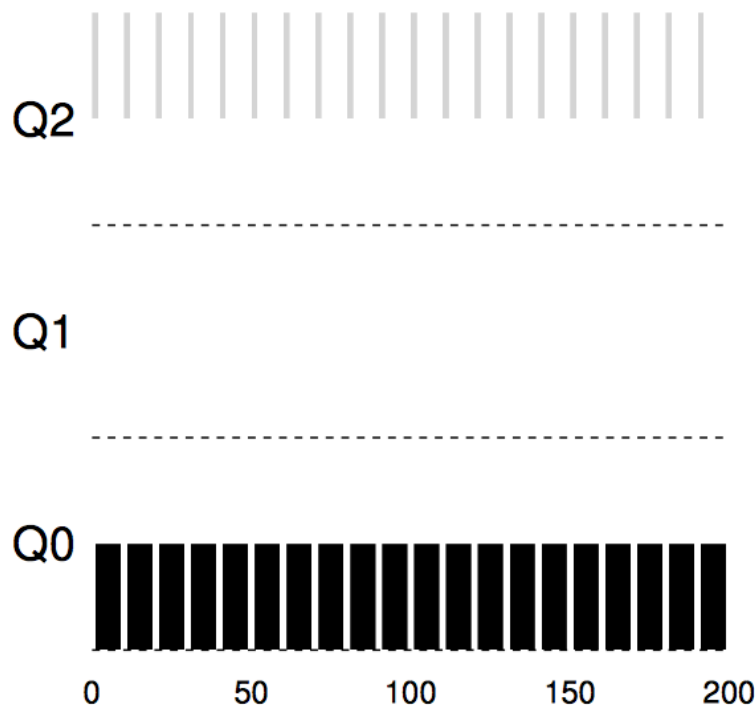
- 例如上述分类中，两类job并不都是turnaround time和response time都需要，而是每类偏重一个方面，这样就容易优化了

Multi-Level Feedback Queue (MLFQ)

- 如何改变优先级
 - **Rule 3:** 任务刚开始时放在最高优先级
 - **Rule 4a:** 如果任务运行时使用了一个完整的时间片, 优先级减低一级 (*i.e., it moves down one queue*). 【针对CPU-bound任务】
 - **Rule 4b:** 如果任务在时间片结束之前提前让出CPU, 则不改变优先级. 【针对interactive任务】

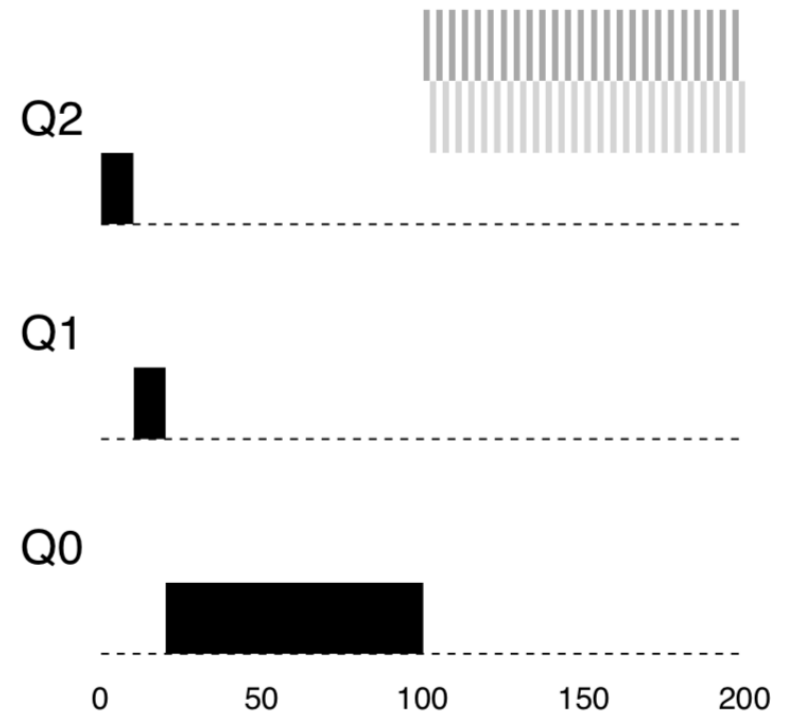
Multi-Level Feedback Queue (MLFQ)

- I/O密集型的任务
 - 按照Rule 4b, 由于I/O阻塞, 会提前释放CPU, 所以一直留在高优先级



Multi-Level Feedback Queue (MLFQ)

- MLFQ问题总结
 - 1. CPU-bound型任务的饥饿问题
 - 如果交互式任务数量较多，它们会交替使用CPU
 - 低优先级的CPU-bound型任务可能饥饿（调度算法一定要避免的灾难）
 - *系统工作，首先要考虑避免灾难，然后才是优化性能

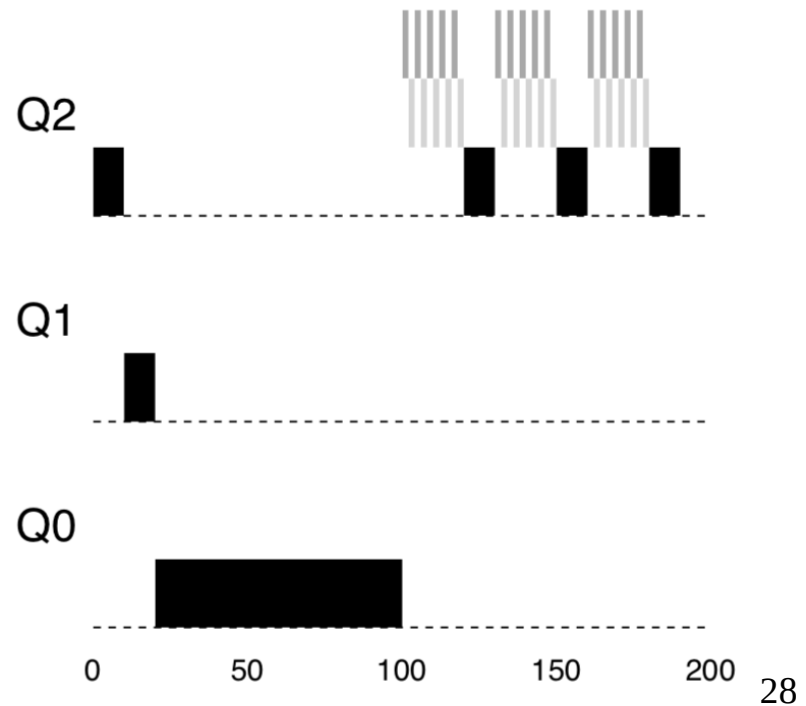


Multi-Level Feedback Queue (MLFQ)

- MLFQ问题总结
 - 1. 饥饿问题
 - 2. 用户会针对性修改程序，在时间片结束前进行一次没有意义的**I/O**（访问一个不需要的文件），这样可以一直停留在最高优先级
 - **Client**和**Server**之间的欺诈、对抗
 - 3. 程序**workload**改变模式（计算->交互），没有机会上升优先级

Multi-Level Feedback Queue (MLFQ)

- 优先级提升 (Priority Boost)
 - **Rule 5:** 一段时间 S 之后, 将系统中所有任务移到最高优先级的队列
 - 解决了两个问题
 - 饥饿
 - **Workload**模式改变 (重新做人的机会)



Multi-Level Feedback Queue (MLFQ)

- 优先级提升 (Priority Boost)
 - Rule 5中的time period S 应该如何设置？
 - 如果过大，没有起到效果，长任务还是可能饥饿；
 - 如果过小，交互式任务的性能可能受到影响；
 - 需要black magic才能做到（与两类任务的比例动态相关）
 - 这类问题在Systems领域叫做voo-doo constants
 - 西非的伏都教，信奉巫术
 - Ousterhout's Law: 在设计算法时尽量避免引入voo-doo constants
 - 这也是为什么简单的方法在系统中更加易用的原因之一（如LRU）

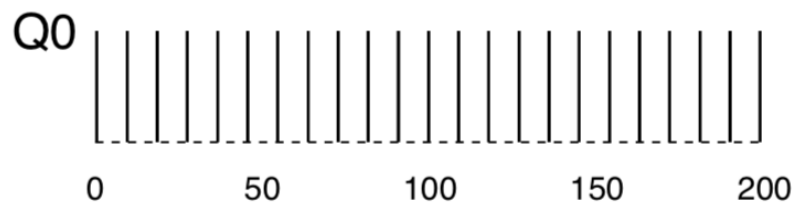
Multi-Level Feedback Queue (MLFQ)

- 优先级提升 (Priority Boost)
 - 仍然没有解决问题2（用户欺诈，伪装自己，在让出时间片前做一次无意义的I/O，以停留在高优先级）



Job A, 欺诈, 占用**99%**的CPU

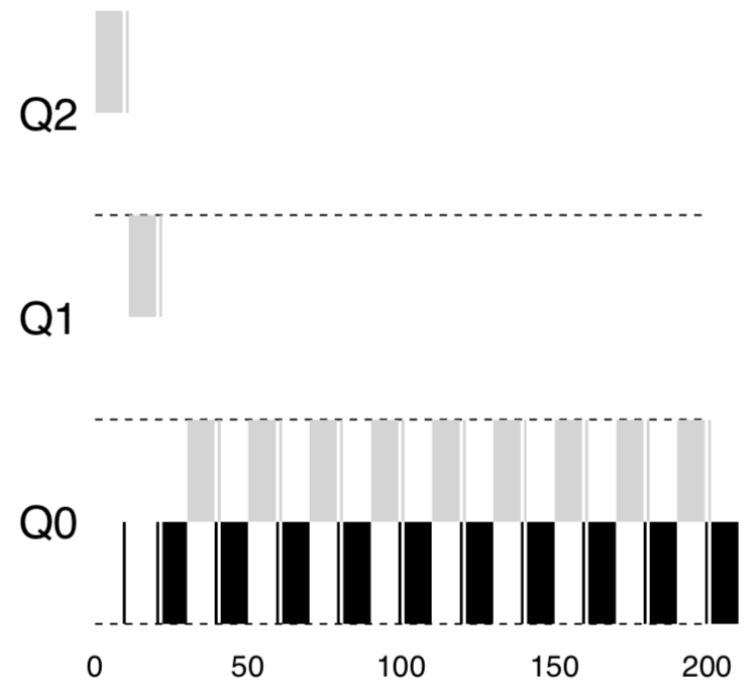
Q1



Job B, 老实, 占用**1%**的CPU

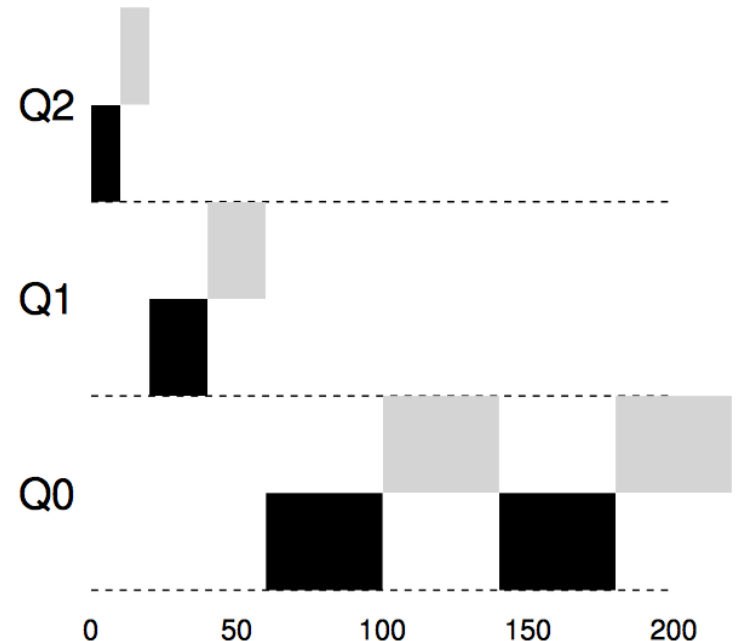
Multi-Level Feedback Queue (MLFQ)

- Better Accounting
 - **Rule 4:** 当一个任务累计使用大量的CPU之后，优先级也会下降 (i.e., it moves down one queue).
 - 占用CPU总数达到一定量，就下降一个优先级
 - 交互式Job占用CPU少，所以应该优先级高，能够达到设计目标
 - 比提前让出时间片更合理，不容易被骗



Multi-Level Feedback Queue (MLFQ)

- MLFQ参数调优
 - Queue数量、每个Q的time slice大小、所有任务回到最高级的周期
 - 例如，上级的Q用小的time slice，下级的Q用大的time slice
 - Interactive job: 减小 response time
 - CPU-bound job: 减小 context switch的开销



Multi-Level Feedback Queue (MLFQ)

- MLFQ参数调优
 - Solaris MLFQ实现—Time-Sharing scheduling class (TS)
 - 一系列参数可以调整
 - 例如队列数量默认是60个
 - 时间片从20ms到几百ms，甚至1s
 - FreeBSD scheduler采用公式计算任务的优先极

Multi-Level Feedback Queue (MLFQ)

- **MLFQ Summary:**

- **Rule 1:** 完全按照优先级.
- **Rule 2:** 优先级相同则使用Round Robin.
- **Rule 3:** 任务最开始放在最高优先级的队列中.
- **Rule 4:** 当任务使用**CPU**达到一定时长后, 降低优先级.
- **Rule 5:** 一段时间**S**之后, 将所有任务移到最高优先级.

复杂系统设计

- 不求最好，但求无灾

Outline

- CPU调度基本策略
- Multi-Level Feedback Queue
- **Proportional Share**
- 多核CPU调度

Proportional Share

- Proportional share scheduler
- 也叫Fair-share scheduler
- 不追求优化turnaround time或response time, 而是保障每个任务得到一定比例的CPU时间
- 一个典型算法是lottery scheduling (Waldspurger and Weihl, 1994)

Lottery Scheduling

- 每个任务持有一定的**tickets**, **schduler**生成随机数, 落在哪个任务的范围内, 就调度哪个任务
- **Tickets**比例决定优先级
 - 例如**A**有**75**个**ticket**, **B**有**25**个**ticket**

Here is an example output of a lottery scheduler's winning tickets:

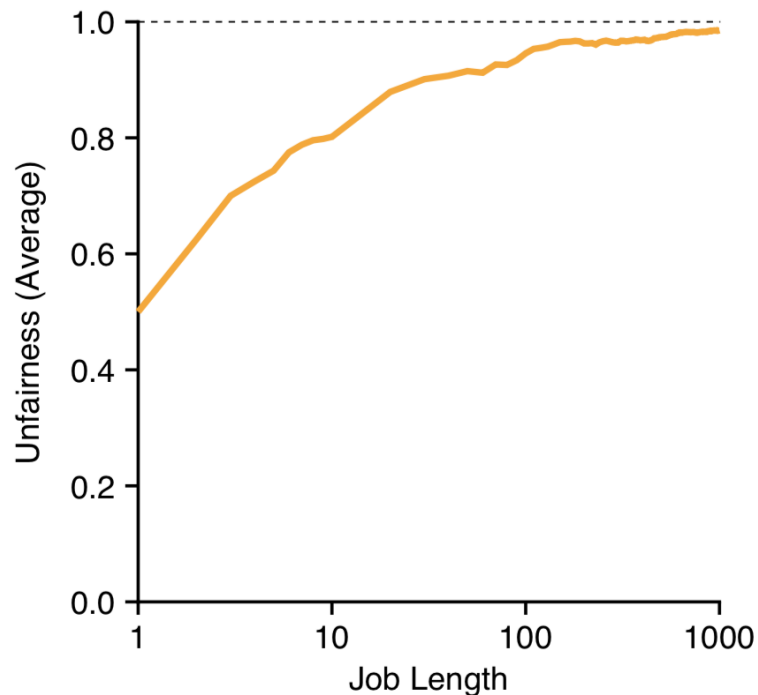
63 85 70 39 76 17 29 41 36 39 10 99 68 83 63 62 43 0 49 49

Here is the resulting schedule:

A B A A B A A A A A A B A B A A A A A

Lottery Scheduling

- 衡量Fairness- Unfairness指数U
 - 任务中第一个完成的时间除以第二个完成时间
 - 例如第一个任务时间10完成，第二个20完成，那么 $U=0.5$
 - 对于基本上同时到达的任务，完全公平的情况下，应该 $U=1$



两个Job，任务越长，
U越接近于1

Lottery Scheduling

- 为什么不使用确定性的调度方法？
 - lottery scheduling不需要维护global state，对于每个job都通过生成随机数调度

Lottery Scheduling

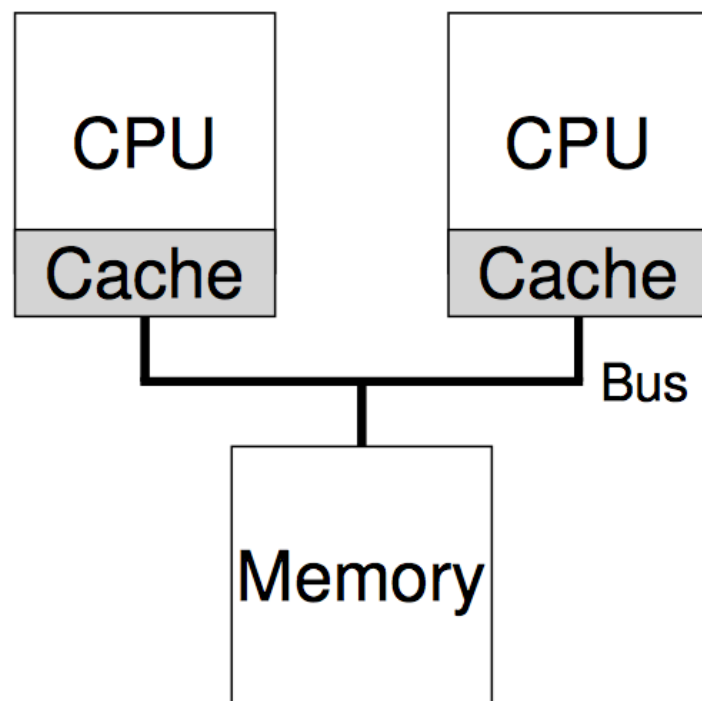
- 随机算法的好处
 - 避免一些特殊情况的问题（例如LRU在cyclic-sequential workloads下命中率低的问题）
 - 简单，轻量级
 - 性能好，快速
- 深入理解RANDOM算法
 - 冷热数据均等几率淘汰
 - 但热数据重新进入概率高很多
 - 看似公平随机，实际更有利于热数据（达到设计目的）

Outline

- CPU调度基本策略
- Multi-Level Feedback Queue
- Proportional Share
- 多核CPU调度

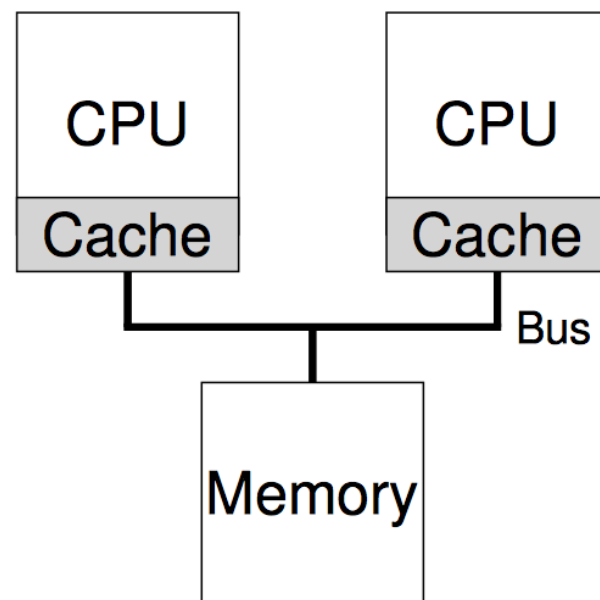
多核CPU调度

- 多核CPU的新问题：Cache一致性问题
 - 核1、核2都cache同一份数据
 - 核1修改数据
 - 核2读数据会不一致
 - 解决：
 - **bus snooping** (总线监听), 在bus监控cache行为, 做相应处理
 - 增加性能开销



多核CPU调度

- bus snooping (总线监听)
 - MESI方案（针对cacheline）
 - **M (modified)**: cl经过修改，是系统唯一正确版本，其他CPU cache中该数据失效(->[I])，从这个cache读取
 - **E (exclusive)**: cache中唯一与主存一致的数据，其他CPU cache中该数据失效(->[I])，从该cache/主存读取；写操作[E]->[M]
 - **S (share)**: 多个cache一致，如果写则write through，改为[E]
 - **I (invalidate)**: 失效，当作没有



多核CPU调度

- 多线程原子性问题
 - 加锁，但会带来性能开销
 - 线程可能被调度到的核越多，冲突越大，性能就越差

```
1  typedef struct __Node_t {
2      int          value;
3      struct __Node_t *next;
4  } Node_t;
5
6  int List_Pop() {
7      Node_t *tmp = head;          // remember old head ...
8      int value   = head->value;    // ... and its value
9      head        = head->next;     // advance head to next pointer
10     free(tmp);                   // free old head
11     return value;                // return value at head
12 }
```

不加锁的话，两个线程同时拿到**head**，都做删除，会
double free

多核CPU调度

- **Cache Affinity** (密切关系, 亲近)
 - 一个进程连续调度到同一个CPU核上, **cache**和**TLB**都有效, 性能会比较好
 - 如果调度到不同的核上, 则**cache**资源浪费

多核CPU调度

- **Single-Queue Multiprocessor Scheduling or SQMS**
 - 一个大队列，哪个核闲置，就调度一个任务
 - 优点: **Load balance**
 - 缺点: **LOCK**开销大; **cache affinity**差, 性能差

CPU 0	A	E	D	C	B	... (repeat) ...
CPU 1	B	A	E	D	C	... (repeat) ...
CPU 2	C	B	A	E	D	... (repeat) ...
CPU 3	D	C	B	A	E	... (repeat) ...

多核CPU调度

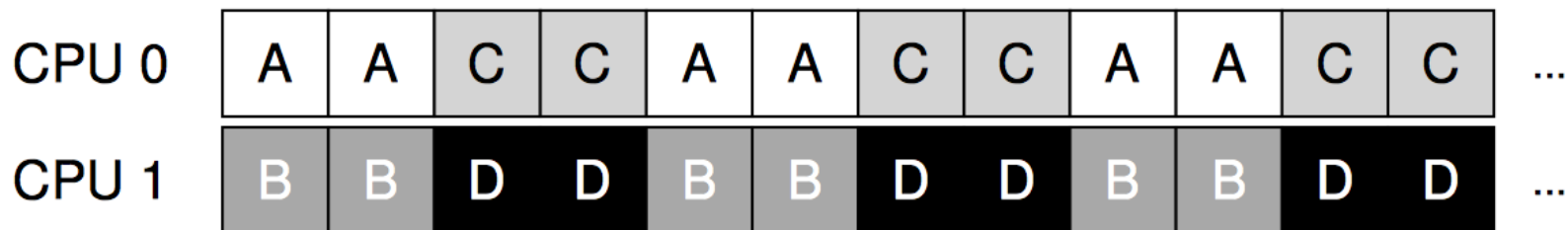
- **Single-Queue Multiprocessor Scheduling or SQMS**

- 改进:
 - 尽量调度到一个CPU核上
 - 少数任务切换
- 扩展性不好

CPU 0	A	E	A	A	A	... (repeat) ...
CPU 1	B	B	E	B	B	... (repeat) ...
CPU 2	C	C	C	E	C	... (repeat) ...
CPU 3	D	D	D	D	E	... (repeat) ...

多核CPU调度

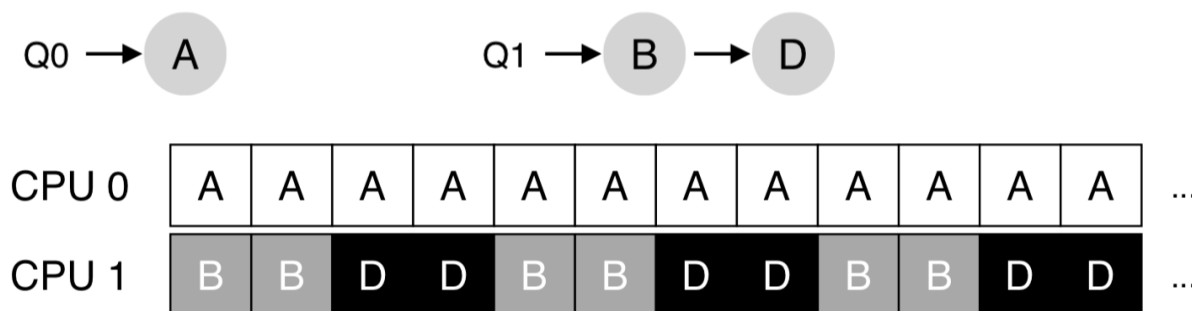
- **Multi-Queue Multiprocessor Scheduling (or MQMS)**
 - 每个CPU核一个独立的队列
 - 优点：性能好（Lock冲突少，cache affinity好）
 - 缺点：load balance
 - 解决：work stealing



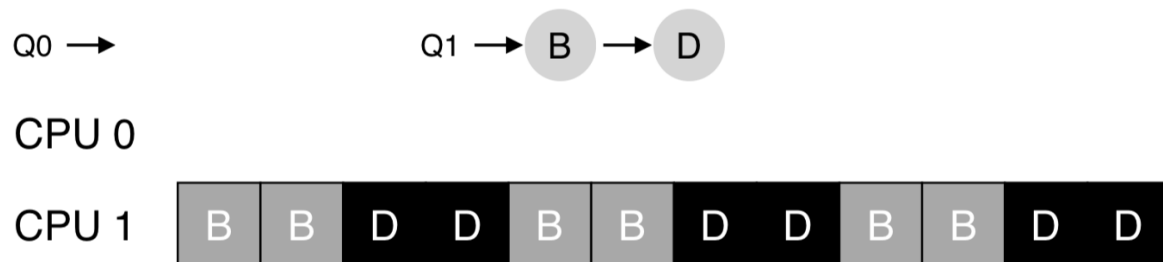
多核CPU调度

- **Multi-Queue Multiprocessor Scheduling (or MQMS)**

- 主要缺点: load imbalance



Job C执行完了，
A的份额是B,D的
两倍

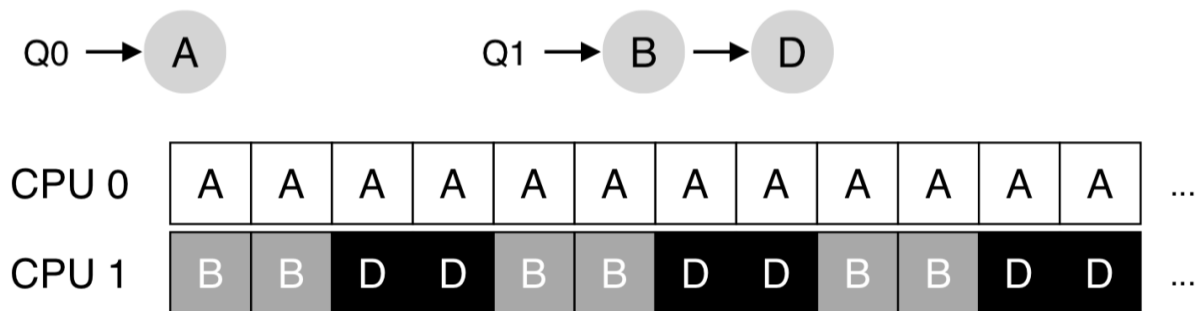


Job A也执行完了
... , CPU 0完全闲置

变形金钢中，Cybertron母星就是被bad CPU scheduling decisions毁了

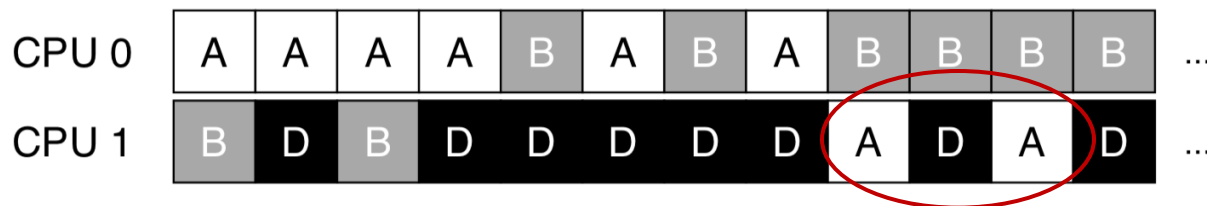
多核CPU调度

- 解决load imbalance的办法: migration
 - 但这种情况下, 单独迁移一个job并不解决问题



- Work stealing

- 不饱和队列从其他队列拿过来一两个job; 但频率提高也会有问题



多核CPU调度

- Linux的多核CPU调度策略
 - 没有一种确定的方案完全优于其他
 - CFS（Completely Fair Scheduler）：多队列
 - 类似于stride scheduler
 - BFS（BF Scheduler）：单队列
 - O(1) Scheduler：多队列
 - 类似于MLFQ

